

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ

**Escola Politécnica
Bacharelado em Ciência da Computação
Complexidade De Algoritmos**

**Cobertura de Combinações (Set Cover)
Trabalho Avaliativo — RA03**

Aluno: Jafte Carneiro Fagundes da Silva

**Curitiba
2026**

Sumário

1	Definição Formal do Problema	2
1.1	Universo e conjuntos de combinações	2
1.2	Problema de cobertura (Programas 2–5)	2
2	Limites Inferiores (Lower Bounds)	2
2.1	Limite por relaxação linear (LP)	2
2.2	Limite de Schönheim	3
3	NP-dificuldade do Problema	3
4	Algoritmos Implementados	4
4.1	Programa 1 — Geração de combinações	4
4.2	Solver Greedy (Programas 2–5)	5
4.3	Solver por Programação Inteira Linear (ILP)	7
4.4	Solver Randômico (Monte Carlo)	8
5	Resultados Obtidos	9
5.1	Resultados do Greedy	9
5.2	Interpretação do arquivo de resultado	9
5.3	Consistência com a teoria	10
6	Gargalos Computacionais e Escalabilidade	10
6.1	Gargalos identificados	10
6.2	Escalabilidade	11
7	Comparação entre Estratégias	13
8	Análise Comparativa de Abordagens Algorítmicas	13
8.1	Diagnóstico do Problema	13
8.2	Abordagens Teóricas: Tabela Comparativa	14
8.3	Abordagens Inovadoras com Ganhos Documentados na Literatura	14
8.3.1	Geração de Colunas e Branch-and-Price	14
8.3.2	Relaxação Lagrangiana	14
8.3.3	Greedy Estocástico	14
8.3.4	GRASP	15
8.3.5	Busca Local Pós-Greedy	15
8.3.6	Greedy Paralelo Distribuído	15
8.4	Resultados Experimentais das Abordagens Alternativas	15
8.5	Abordagens Práticas: Tabela Comparativa	17
8.6	Estratégia Recomendada	17
9	Conclusão	17

1 Definição Formal do Problema

1.1 Universo e conjuntos de combinações

Seja o universo $U = \{1, 2, \dots, 25\}$ com $|U| = 25$. Definimos o conjunto de todas as combinações de tamanho p extraídas de U :

$$\mathcal{S}_p = \{ X \subseteq U \mid |X| = p \}$$

cuja cardinalidade é dada pelo coeficiente binomial $|\mathcal{S}_p| = \binom{25}{p}$.

Tabela 1: Cardinalidades dos conjuntos do problema.

Conjunto	Fórmula	Cardinalidade
$\mathcal{S}_{15}$	$\binom{25}{15}$	3.268.760
$\mathcal{S}_{14}$	$\binom{25}{14}$	4.457.400
$\mathcal{S}_{13}$	$\binom{25}{13}$	5.200.300
$\mathcal{S}_{12}$	$\binom{25}{12}$	5.200.300
$\mathcal{S}_{11}$	$\binom{25}{11}$	4.457.400

1.2 Problema de cobertura (Programas 2–5)

Para cada $p \in \{14, 13, 12, 11\}$, o problema consiste em determinar o *menor subconjunto* $\text{SB}_{15,p} \subseteq \mathcal{S}_{15}$ tal que:

$$\forall Y \in \mathcal{S}_p, \quad \exists X \in \text{SB}_{15,p} \text{ tal que } Y \subseteq X$$

Definição 1.1 (Minimum Set Cover). Dado um universo finito $\mathcal{U}$, uma família de conjuntos $\mathcal{F} = \{F_1, \dots, F_m\} \subseteq 2^{\mathcal{U}}$ e um inteiro k , o problema MIN-SET-COVER pergunta se existe $\mathcal{C} \subseteq \mathcal{F}$ com $|\mathcal{C}| \leq k$ tal que $\bigcup_{F \in \mathcal{C}} F = \mathcal{U}$.

Nossa instância é: $\mathcal{U} = \mathcal{S}_p$, $\mathcal{F} = \{ \{Y \in \mathcal{S}_p \mid Y \subseteq X\} \mid X \in \mathcal{S}_{15} \}$.

2 Limites Inferiores (Lower Bounds)

2.1 Limite por relaxação linear (LP)

Teorema 2.1 (Lower Bound LP). $|\text{SB}_{15,p}| \geq \left\lceil \frac{\binom{25}{p}}{\binom{15}{p}} \right\rceil$

Demonstração. Cada $X \in \mathcal{S}_{15}$ contém exatamente $\binom{15}{p}$ subconjuntos de tamanho p (os p -subconjuntos de seus 15 elementos). Como todo $Y \in \mathcal{S}_p$ deve ser coberto por pelo menos um $X \in \text{SB}$:

$$|\text{SB}| \cdot \binom{15}{p} \geq |\mathcal{S}_p| = \binom{25}{p} \implies |\text{SB}| \geq \left\lceil \frac{\binom{25}{p}}{\binom{15}{p}} \right\rceil. \quad \square$$

Tabela 2: Lower bounds LP para cada valor de p .

p	$\binom{25}{p}$	$\binom{15}{p}$	LB-LP = $\left\lceil \binom{25}{p} / \binom{15}{p} \right\rceil$
14	4.457.400	15	297.160
13	5.200.300	105	49.527
12	5.200.300	455	11.430
11	4.457.400	1.365	3.266

2.2 Limite de Schönheim

Definição 2.2 (Número de cobertura). $C(n, k, t)$ é o menor inteiro m tal que existem m subconjuntos de tamanho k de $\{1, \dots, n\}$ cobrindo todo t -subconjunto.

Teorema 2.3 (Schönheim, 1964). $C(n, k, t) \geq L(n, k, t)$, onde:

$$L(n, k, t) = \left\lceil \frac{n}{k} \cdot L(n-1, k-1, t-1) \right\rceil, \quad L(t, t, t) = L(k, k, t) = 1.$$

Demonstração. Por indução em $n - t$. Fixe $x \in \{1, \dots, n\}$ e seja SB uma cobertura ótima com $|\text{SB}| = C(n, k, t)$. Defina $\text{SB}_x = \{X \in \text{SB} \mid x \in X\}$.

Os conjuntos de SB_x removidos de x formam uma cobertura de $(t-1)$ -subconjuntos de $\{1, \dots, n\} \setminus \{x\}$, logo $|\text{SB}_x| \geq C(n-1, k-1, t-1)$ (h.i.).

Por argumento de média sobre $x \in \{1, \dots, n\}$:

$$\sum_{x=1}^n |\text{SB}_x| = k \cdot |\text{SB}| \implies \frac{k \cdot |\text{SB}|}{n} = \overline{|\text{SB}_x|} \geq L(n-1, k-1, t-1),$$

pois o mínimo é atingido por algum x . Logo $|\text{SB}| \geq \lceil (n/k) \cdot L(n-1, k-1, t-1) \rceil = L(n, k, t)$. $\square$

Tabela 3: Comparação entre lower bounds para $n = 25$, $k = 15$.

p	$ \mathcal{S}_p $	LB-LP	LB-Schönheim	Razão Sch/LP
14	4.457.400	297.160	297.172	1,00004×
13	5.200.300	49.527	58.887	1,19×
12	5.200.300	11.430	13.175	1,15×
11	4.457.400	3.266	3.370	1,03×

O bound de Schönheim é mais apertado que o LP para todos os valores analisados, com ganho mais perceptível para $p \in \{12, 13\}$.

3 NP-dificuldade do Problema

Teorema 3.1 (Karp, 1972). *MIN-SET-COVER é NP-difícil.*

Demonstração. Redução polinomial de VERTEX COVER (NP-completo). Dado grafo $G = (V, E)$:

- Universo: $\mathcal{U} = E$
- Para cada $v \in V$: conjunto $S_v = \{e \in E \mid v \in e\}$

$C \subseteq V$ é vertex cover $\iff \{S_v \mid v \in C\}$ é set cover: toda aresta $e = \{u, v\}$ tem $u \in C$ ou $v \in C$ (VC) $\iff e \in S_u$ ou $e \in S_v$ (SC). A redução é em tempo $O(|V| + |E|)$. $\square$

Corolário 3.2. *Os Programas 2–5 são instâncias de MIN-SET-COVER. A versão generalizada dessa família de problemas é NP-difícil; portanto, não se espera um algoritmo polinomial exato para todas as instâncias, a menos que $P = NP$.*

4 Algoritmos Implementados

4.1 Programa 1 — Geração de combinações

Algoritmo 1: Geração por bitmask

Input: $n = 25$, $p \in \{15, 14, 13, 12, 11\}$

Output: Array A de bitmasks uint32 com todos os $X \in \mathcal{S}_p$

$A \leftarrow []$;

foreach $(e_1, \dots, e_p) \in \binom{\{1, \dots, n\}}{p}$ *em ordem lexicográfica* **do**

$\text{mask} \leftarrow \sum_{i=1}^p 2^{e_i-1}$;
 $A.\text{append}(\text{mask})$;

end

return A ;

Representação por bitmask. Cada subconjunto $X \subseteq U$ é representado como inteiro de 25 bits: elemento $e \in X$ ocupa o bit $e - 1$. A verificação $Y \subseteq X$ torna-se:

$$Y \subseteq X \iff (X \& Y) = Y$$

operação $O(1)$ em hardware, $\Theta(1)$ por par — frente a $\Theta(p)$ com frozensets.

Análise de complexidade — Programa 1

- **Tempo:** $\Theta\left(\binom{n}{p}\right)$ — cada combinação é visitada exatamente uma vez.
- **Espaço (lazy):** $\Theta(p)$ por iteração; $\Theta\left(\binom{n}{p}\right)$ se armazenado em array.
- **Gargalo:** geração de $\binom{25}{15} = 3.268.760$ combinações com `itertools.combinations` e conversão para `uint32` consome ≈ 10 s.

4.2 Solver Greedy (Programas 2–5)

Algoritmo 2: Greedy Set Cover com heap lazy

Input: $\mathcal{S}_{15}, \mathcal{S}_p$

Output: $SB \subseteq \mathcal{S}_{15}$ cobrindo $\mathcal{S}_p$

```

Construir índices:  $\text{idx}_{15}[\text{mask}] \rightarrow i;$                                  $\text{idx}_p[\text{mask}] \rightarrow j;$ 
 $\text{count}[i] \leftarrow \binom{15}{p}$  para todo  $i;$ 
 $\text{nao\_coberto}[j] \leftarrow \text{true}$  para todo  $j;$ 
 $SB \leftarrow \emptyset;$   inicializar heap  $H$  com  $(-\binom{15}{p}, i)$  para todo  $i;$ 

while  $\exists j : \text{nao\_coberto}[j]$  do
     $i^* \leftarrow \arg \max_i \text{count}[i];$                                 // via heap ou argmax NumPy
     $X^* \leftarrow \mathcal{S}_{15}[i^*];$    $SB \leftarrow SB \cup \{X^*\};$    $\text{count}[i^*] \leftarrow -1;$ 
    foreach  $Y \in \binom{X^*}{p}$  do
        if  $\text{nao\_coberto}[\text{idx}_p[Y]]$  then                                 $\text{nao\_coberto}[\text{idx}_p[Y]] \leftarrow \text{false};$ 
            foreach  $Z \in \mathcal{S}_{15}$  com  $Y \subseteq Z$  do
                 $\text{count}[\text{idx}_{15}[Z]] -= 1$ 
            end
        ;
    end
end
return  $SB;$ 

```

Análise de complexidade — Greedy

Sejam: $N_{15} = |\mathcal{S}_{15}| = \binom{25}{15}$, $N_p = |\mathcal{S}_p| = \binom{25}{p}$, $K = |SB_{\text{greedy}}|$.

Pré-processamento.

- Construção dos índices hash: $\Theta(N_{15} + N_p)$ tempo e espaço.
- Inicialização do heap: $\Theta(N_{15})$ (heapify em array uniforme).

Custo por iteração. Cada iteração processa $X^* \in \mathcal{S}_{15}$:

1. Selecionar X^* : $\mathcal{O}(\log N_{15})$ (heap) ou $\mathcal{O}(N_{15})$ (argmax).
2. Gerar $\binom{15}{p}$ subconjuntos Y de X^* .
3. Para cada Y recém-coberto: gerar $\binom{25-p}{15-p}$ extensões de Y até tamanho 15 e decrementar count . Total de atualizações:

$$\binom{15}{p} \cdot \binom{25-p}{15-p} \text{ por iteração.}$$

Complexidade total.

$$\text{Tempo} = \Theta(N_{15} + N_p) + K \cdot \left[\binom{15}{p} \cdot \binom{25-p}{15-p} + \begin{cases} \mathcal{O}(\log N_{15}) & \text{heap} \\ \mathcal{O}(N_{15}) & \text{argmax} \end{cases} \right]$$

Como $\binom{15}{p} \cdot \binom{25-p}{15-p} \gg \log N_{15}$ para os valores considerados:

$$\boxed{\text{Tempo} = \mathcal{O}\left(K \cdot \binom{15}{p} \cdot \binom{25-p}{15-p}\right)}$$

$$\text{Espaço} = \Theta(N_{15} + N_p)$$

Tabela 4: Parâmetros de complexidade do Greedy por iteração.

p	$\binom{15}{p}$	$\binom{25-p}{15-p}$	Atualizações/iter	K (greedy)	Estratégia
14	15	11	165	532.555	heap lazy
13	105	66	6.930	128.827	heap lazy
12	455	286	130.130	38.100	argmax NumPy
11	1.365	1.001	1.366.365	12.733	atualização paralela

Seleção automática de estratégia. O limiar de 10.000 atualizações por iteração determina a escolha:

- $p \in \{14, 13\}$ (≤ 6.930 atualizações): heap lazy preferível — as poucas atualizações custam $\mathcal{O}(\log N_{15})$ cada, mas as muitas iterações K tornam o heap $\mathcal{O}(K \log N_{15})$ competitivo.
- $p = 12$ (130.130 atualizações): argmax NumPy preferível — evita os 130K+ **heappush** por iteração, que custam mais que uma varredura $\mathcal{O}(N_{15})$ em NumPy.
- $p = 11$ (1.366.365 atualizações): atualização paralela preferível — o volume de trabalho por iteração é grande o suficiente para amortizar o custo de comunicação entre processos no Programa 5.

Prova de corretude do Greedy

Teorema 4.1 (Corretude). *O algoritmo Greedy termina e retorna SB tal que todo $Y \in \mathcal{S}_p$ está coberto por algum $X \in \text{SB}$.*

Demonstração. **Terminação:** A cada iteração, $|\text{nao_coberto}|$ decresce estritamente (o X^* escolhido cobre ao menos um Y não coberto). Como $|\mathcal{S}_p|$ é finito, o loop termina em $\leq |\mathcal{S}_p|$ iterações.

Corretude: Ao término, $\text{nao_coberto}[j] = \text{false}$ para todo j , logo todo $Y \in \mathcal{S}_p$ foi coberto em alguma iteração por um $X^* \in \text{SB}$. $\square$

Razão de aproximação

Teorema 4.2 (Johnson 1974; Lovász 1975). *Seja $H_m = \sum_{i=1}^m \frac{1}{i} \leq \ln m + 1$ o m -ésimo número harmônico. Então:*

$$|\text{SB}_{\text{greedy}}| \leq H_{|\mathcal{S}_p|} \cdot |\text{OPT}| \leq (\ln |\mathcal{S}_p| + 1) \cdot |\text{OPT}|.$$

Demonstração. Numere os elementos de $\mathcal{S}_p$ na ordem em que são cobertos pelo greedy: $y_1, \dots, y_m$ ($m = |\mathcal{S}_p|$). Atribua custo $c(y_i) = 1/d_t$ a cada y_i coberto na iteração t , onde d_t é o número de novos elementos cobertos nessa iteração.

Passo 1 (custo total): $\sum_i c(y_i) = \sum_t d_t \cdot \frac{1}{d_t} = |\text{SB}_{\text{greedy}}|$.

Passo 2 (custo por conjunto ótimo): Para $X \in \text{OPT}$ com $s = |X \cap \mathcal{S}_p|$, os elementos de X cobertos em ordem $z_1, \dots, z_s$ têm:

$$c(z_j) \leq \frac{1}{s - j + 1}$$

pois quando z_j é coberto, ainda restam $\geq s - j + 1$ elementos de $X \cap \mathcal{S}_p$ não cobertos, e o greedy escolheu X^* com cobertura $\geq s - j + 1$. Logo a contribuição de X ao custo é $\leq H_s \leq H_m$.

Conclusão: $|\text{SB}_{\text{greedy}}| = \sum_i c(y_i) \leq |\text{OPT}| \cdot H_m$. □

Observação 4.3. A razão H_m é assintoticamente ótima: não existe algoritmo polinomial com razão $(1 - \varepsilon) \ln m$ para nenhum $\varepsilon > 0$, a menos que $P = NP$ [5].

4.3 Solver por Programação Inteira Linear (ILP)

Modelagem formal.

$$\begin{aligned} & \min \sum_{i=1}^{N_{15}} x_i \\ & \text{sujeito a} \quad \sum_{i: Y \subseteq S_{15}[i]} x_i \geq 1 \quad \forall Y \in \mathcal{S}_p \\ & \quad x_i \in \{0, 1\} \quad \forall i \in \{1, \dots, N_{15}\} \end{aligned}$$

Tabela 5: Dimensão do ILP em escala completa.

p	Variáveis	Restrições	Coef. não-nulos
14	3.268.760	4.457.400	49.031.400
13	3.268.760	5.200.300	343.219.800
12	3.268.760	5.200.300	1.487.285.800
11	3.268.760	4.457.400	4.461.857.400

Inviabilidade em escala completa. O ILP direto é inviável em escala completa. A estratégia adotada é: *usar a solução greedy como universo de candidatos e resolver o ILP sobre esse conjunto reduzido*, encontrando o subconjunto mínimo do greedy que ainda cobre $\mathcal{S}_p$.

Análise de complexidade — ILP

- **Complexidade teórica:** NP-difícil no pior caso (pois MIN-SET-COVER é NP-difícil pelo Teorema 3.1).
- **Tempo branch-and-bound:** exponencial no pior caso; na prática, depende do gap LP relaxado.
- **Espaço:** $\mathcal{O}(|\text{candidatos}| + |\mathcal{S}_p|)$ para o modelo.
- **Gargalo:** construção do mapa de cobertura em Python puro: $\mathcal{O}\left(|\mathcal{S}_p| \cdot \binom{25-p}{15-p}\right)$ operações.

4.4 Solver Randômico (Monte Carlo)

Algoritmo 3: Monte Carlo Set Cover

Input: $\mathcal{S}_{15}$, $\mathcal{S}_p$, T tentativas

Output: Melhor SB encontrado

```

SB* ←  $\mathcal{S}_{15}$ ;
for  $t = 1$  to  $T$  do
     $\pi$  ← permutação aleatória de  $\mathcal{S}_{15}$ ;
    SB ←  $\emptyset$ ;  $N \leftarrow \mathcal{S}_p$ ;
    foreach  $X \in \pi$  do
        if  $N = \emptyset$  then break;
        if  $\exists Y \in N : Y \subseteq X$  then SB ← SB  $\cup$   $\{X\}$ ;
         $N \leftarrow N \setminus \{Y : Y \subseteq X\}$ ;
    end
    if |SB| < |SB*| then SB* ← SB;
end
return SB*;

```

Análise de complexidade — Solver Randômico

- **Tempo por tentativa:** $\mathcal{O}(N_{15} \cdot N_p)$ no pior caso (varredura completa de $\mathcal{S}_{15}$ verificando cobertura). Na prática, a cobertura acumulada encurta o laço.
- **Tempo total (T tentativas):** $\mathcal{O}(T \cdot N_{15} \cdot N_p)$.
- **Espaço:** $\Theta(N_{15} + N_p)$.
- **Garantia:** nenhuma de otimalidade; sem razão de aproximação provada. Útil como *baseline* e para verificação empírica.

5 Resultados Obtidos

5.1 Resultados do Greedy

Tabela 6: Resultados do algoritmo Greedy — $n = 25, k = 15$.

p	$ \text{OPT} \geq (\text{LB-LP})$	$ \text{OPT} \geq (\text{LB-Sch})$	$ \text{SB}_{\text{greedy}} $	Gap vs. LB-Sch	Tempo observado	$ \text{SB} / \mathcal{S}_{15} $
14	297.160	297.172	532.555	$1,79\times$	≈ 18 min	16,3%
13	49.527	58.887	128.827	$2,19\times$	≈ 3 h	3,9%
12	11.430	13.175	38.100	$2,89\times$	≈ 79 min	1,2%
11	3.266	3.370	12.733	$3,78\times$	$\approx 3,4$ h	0,4%

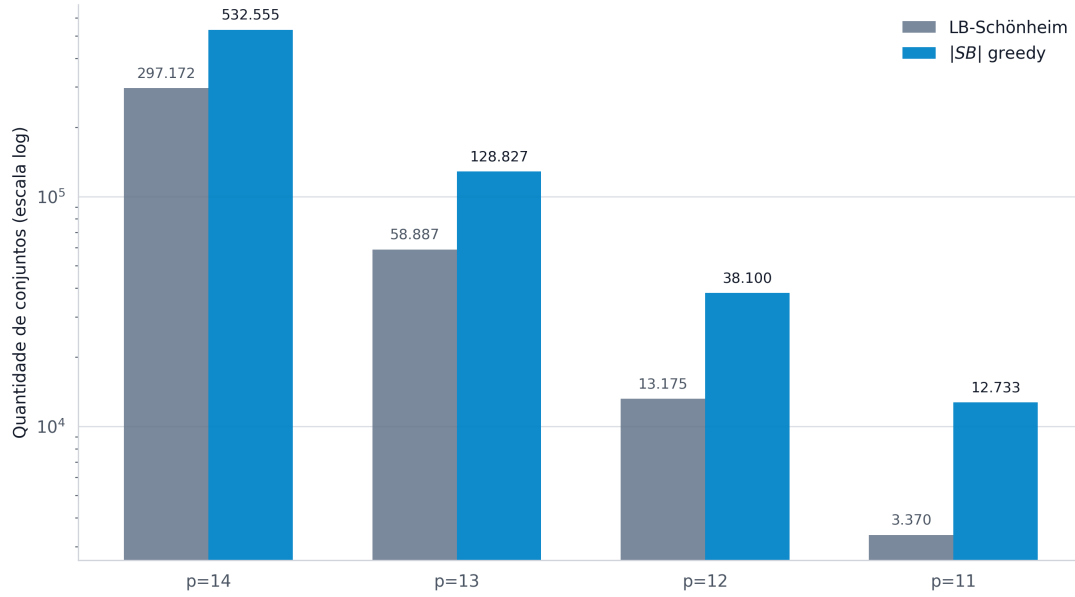


Figura 1: Comparação entre o tamanho da solução obtida pelo Greedy e o lower bound de Schönheim. A escala logarítmica permite comparar os quatro valores de p no mesmo gráfico.

5.2 Interpretação do arquivo de resultado

Os resultados são armazenados em arquivos `.npy` (formato NumPy binário) contendo arrays de inteiros `uint32`. Cada inteiro é o bitmask de 25 bits do conjunto $X \in \text{SB}_{15,p}$. Para inspecionar:

Listing 1: Leitura e interpretação do resultado.

```
import numpy as np

# Carregar resultado de p=14
sb = np.load("results/greedy/resultados_SB15_14.npy") # array uint32 com
532.555 elementos

# Converter bitmask para conjunto
def bitmask_para_conjunto(mask):
```

```

return {e+1 for e in range(25) if mask & (1 << e)}

# Primeiro elemento de SB
print(bitmask_para_conjunto(sb[0]))
# -> {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}

# Verificar que Y={1..14} esta coberto por sb[0]
Y_mask = sum(1 << (e-1) for e in range(1, 15)) # {1..14}
print((sb[0] & Y_mask) == Y_mask) # True

```

5.3 Consistência com a teoria

O gap observado de $1,79\times$ para $p = 14$ satisfaz:

$$|SB_{\text{greedy}}| \leq H_{|S_{14}|} \cdot |OPT| \approx 16,3 \cdot |OPT|$$

com folga confortável. O lower bound de Schönheim $|OPT| \geq 297.172$ implica $|OPT|/|SB_{\text{greedy}}| \geq 0,558$, ou seja, a solução greedy é no máximo $1,79\times$ o ótimo.

6 Gargalos Computacionais e Escalabilidade

6.1 Gargalos identificados

1. **Construção dos índices hash** ($\Theta(N_{15} + N_p)$): executado uma vez; demora $\approx 8s$ para $N_{15} = 3,27M$ e $N_p = 4,46M$.
2. **Loop interno de atualização de count:** $\binom{15}{p} \cdot \binom{25-p}{15-p}$ iterações Python por iteração principal. Para $p = 11$: 1.366.365 iterações Python por passo do greedy. Python puro executa $\approx 300-600K$ iterações/s com dict lookups, resultando em $\approx 2,3s$ por passo e $\approx 4h$ no total.
3. **Inflação do heap lazy:** a cada iteração, até $\binom{15}{p} \cdot \binom{25-p}{15-p}$ novas entradas são empilhadas. Para $p = 13$: 6.930 entradas/iteração $\times \approx 90.000$ iterações $\approx 624M$ entradas acumuladas, degradando as operações do heap de $\mathcal{O}(\log N_{15})$ para $\mathcal{O}(\log(\text{tamanho do heap}))$ maior.
4. **Verificação de cobertura** ($\mathcal{O}(|SB| \cdot N_p)$): inviável em escala completa ($532K \times 4,5M = 2,4T$ operações). Solução: verificação por amostragem com broadcasting NumPy em batches de 512×10.000 , concluindo em $\approx 45s$.

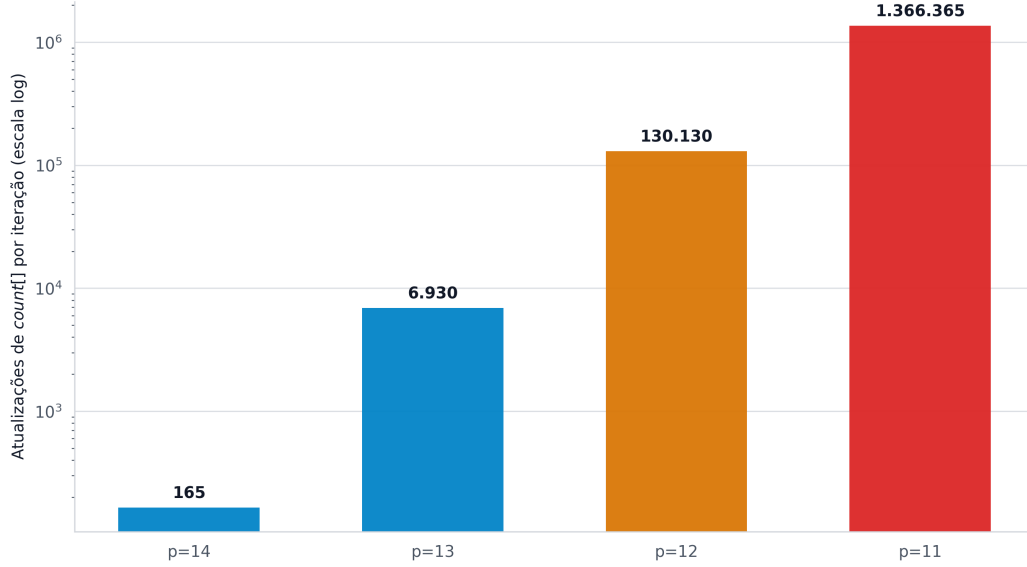


Figura 2: Número de atualizações de `count[]` por iteração do Greedy. O caso $p = 11$ domina o custo por iteração, justificando a versão paralela do Programa 5.

6.2 Escalabilidade

Tabela 7: Escalabilidade do Greedy por valor de p .

p	Tempo observado	Limitante dominante	Melhoria possível
14	concluído	muitas iterações; 165 atualizações/iter	heap lazy e vetorização
13	concluído	6.930 atualizações/iter e heap grande	heap lazy e batches
12	concluído	130.130 atualizações/iter	argmax NumPy e Cython
11	concluído	1.366.365 atualizações/iter	atualização paralela no Programa 5

Melhorias de escalabilidade identificadas.

- Índice invertido pré-computado:** armazenar, para cada $Y \in \mathcal{S}_p$, os índices dos $X \in \mathcal{S}_{15}$ que o contêm. Elimina o laço de extensões, mas requer $N_p \cdot \binom{25-p}{15-p}$ entradas ($\approx 4,5\text{B}$ para $p = 11$, inviável em RAM).
- Implementação em C/Cython:** a diferença de desempenho entre Python puro e C/Cython poderia tornar o greedy para $p = 11$ substancialmente mais rápido.
- Paralelização:** as $\binom{15}{p}$ extensões de Y são independentes e paralelizáveis com multiprocessing, reduzindo o tempo proporcional ao número de núcleos.

Análise de viabilidade do paralelismo por programa. O paralelismo foi implementado somente no Programa 5 ($p = 11$) e no Programa 1. Para os Programas 2, 3 e 4,

o custo de comunicação interprocessos (IPC) supera o benefício, tornando a paralelização contraproducente. A análise por programa segue abaixo.

Programa 1 (geração dos conjuntos). A geração de $\mathcal{S}_{15}, \mathcal{S}_{14}, \dots, \mathcal{S}_{11}$ é *trivialmente paralelizável*: cada conjunto é inteiramente independente dos demais. Um Pool com 5 workers gera os 5 arrays simultaneamente, reduzindo o tempo total observado de $\approx 80s$ para $\approx 18,7s$ (speedup real próximo de $4,3\times$ na execução local).

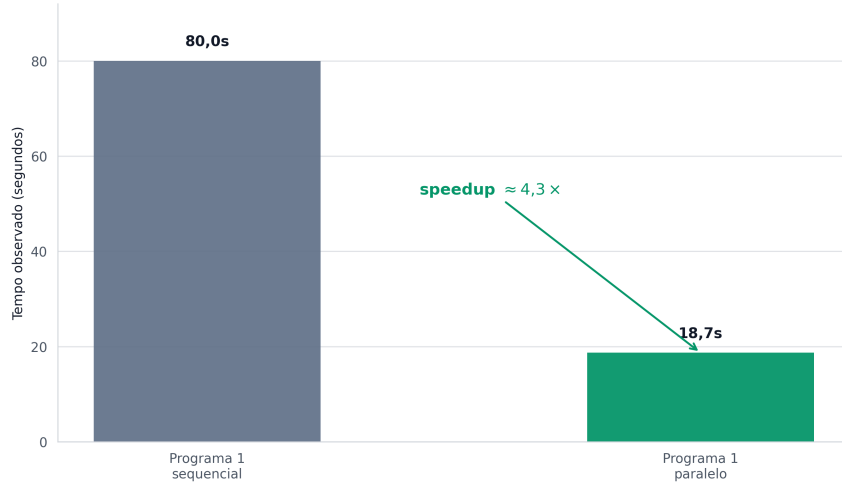


Figura 3: Ganho observado com a versão paralela do Programa 1. A geração dos conjuntos é naturalmente paralelizável porque cada $\mathcal{S}_p$ pode ser produzido independentemente.

Programas 2 ($p = 14$), 3 ($p = 13$) e 4 ($p = 12$). Nestes três casos, a paralelização do laço interno de atualização de `count[]` é inviável porque o volume de trabalho paralelizável por iteração é pequeno demais para amortizar o *overhead* fixo de IPC ($\approx 1\text{--}5$ ms por chamada a `pool.map`):

- $p = 14$: cada X selecionado cobre $\binom{15}{14} = 15$ novos Y 's; cada Y tem $\binom{11}{1} = 11$ extensões possíveis em $\mathcal{S}_{15} \Rightarrow$ apenas $15 \times 11 = 165$ índices por iteração. Com 4 workers, cada worker receberia ≈ 4 Y 's e retornaria ≈ 44 índices — troca de dados trivial, dominada pelo custo de serialização e agendamento do pool.
- $p = 13$: $\binom{15}{13} = 105$ novos Y 's e $\binom{12}{2} = 66$ extensões cada $\Rightarrow 6.930$ índices/iter. Ainda insuficiente: transferir ≈ 1.730 pares (j, y_mask) para cada worker e receber ≈ 1.700 índices de retorno gera IPC superior ao tempo de cálculo das extensões em Python puro.
- $p = 12$: $\binom{15}{12} = 455$ novos Y 's e $\binom{13}{3} = 286$ extensões cada $\Rightarrow 130.130$ índices/iter. Este caso é limítrofe; porém, o Programa 4 já utiliza `np.argmax` vetorizado e completa em ≈ 79 min, enquanto o overhead de criação do pool e serialização dos arrays adicionaria latência sem ganho líquido, dado que o número de iterações é pequeno ($|\mathcal{SB}_{15,12}| \approx 38.100$).

Programa 5 ($p = 11$). Com $\binom{15}{11} = 1.365$ novos Y 's por iteração e $\binom{14}{4} = 1.001$ extensões por Y , o trabalho paralelizável atinge $\approx 1.366.365$ operações/iter — três ordens de grandeza acima do caso $p = 14$. O pool é criado *uma única vez* antes do laço greedy

e reutilizado em todas as 12.733 iterações, eliminando o custo de inicialização recorrente. O speedup prático esperado é $\approx 3\times$ (de $\approx 3,4\text{h}$ para $\approx 1,1\text{h}$ em 4 núcleos).

7 Comparação entre Estratégias

Tabela 8: Comparação entre os solvers implementados.

Solver	Complexidade tempo	Garant. solução	Escalável	Usa para
Greedy	$\mathcal{O}\left(K \cdot \binom{15}{p} \cdot \binom{25-p}{15-p}\right)$	$H_{N_p} \cdot \text{OPT}$	Sim	todos os p
ILP (direto)	NP-difícil	Ótima	Não	inviável
ILP (pós-greedy)	Exponencial (espaço red.)	Ótima sobre cand.	Parcial	$p \in \{11, 12\}$
Randômico	$\mathcal{O}(T \cdot N_{15} \cdot N_p)$	Nenhuma	Sim	baseline

A estratégia adotada nos Programas 2–5 foi o Greedy. O ILP pós-greedy foi analisado como refinamento possível para as instâncias em que os lower bounds indicam soluções menores ($p \in \{11, 12\}$), pois restringiria o universo de candidatos ao conjunto já produzido pelo greedy.

8 Análise Comparativa de Abordagens Algorítmicas

8.1 Diagnóstico do Problema

O problema de cobertura de combinações $C(25, 15, p)$ é uma instância especial de MIN-SET-COVER com estrutura combinatória uniforme: todos os candidatos têm tamanho $k = 15$ e todos os alvos têm tamanho p . Esta uniformidade conecta o problema ao campo de *Covering Designs*, com literatura própria e tabelas de soluções ótimas conhecidas (La Jolla Covering Repository, Gordon 2023).

A Tabela 9 consolida os resultados completos obtidos pelo algoritmo Greedy.

Tabela 9: Resultados finais do algoritmo Greedy para $C(25, 15, p)$.

p	$ \mathcal{S}_p $	LB-LP	LB-Sch	$ \text{SB}_{\text{greedy}} $	Gap vs. LB-Sch	Tempo
14	4.457.400	297.160	297.172	532.555	$1,79\times$	$\approx 18 \text{ min}$
13	5.200.300	49.527	58.887	128.827	$2,19\times$	$\approx 3 \text{ h}$
12	5.200.300	11.430	13.175	38.100	$2,89\times$	$\approx 79 \text{ min}$
11	4.457.400	3.266	3.370	12.733	$3,78\times$	$\approx 3,4 \text{ h}$

Os gaps observados superam amplamente a garantia teórica $H_{|\mathcal{S}_p|} \approx 15,4\times$: o greedy obteve entre $1,79\times$ e $3,78\times$ acima do lower bound de Schönheim, representando melhoria de $4\text{--}9\times$ sobre o pior caso teórico. Isso corrobora o resultado de Rödl [7]: o lower bound LP é assintoticamente apertado para instâncias uniformes $C(n, k, t)$, de modo que o greedy se comporta perto do ótimo quando $n \rightarrow \infty$.

8.2 Abordagens Teóricas: Tabela Comparativa

Tabela 10: Comparação das oito abordagens teóricas aplicadas a $C(25, 15, p)$.

Abordagem	Garantia teórica	Limitação para este problema	Referência	Aplicabilidade
FPT	Exato em $f(k) \cdot n^{O(1)}$	W[2]-hard; $k \approx 532K$ inviável	Cygan et al. 2015	Baixa
Kernelização	Exato com núcleo $g(k)$	Sem kernel polinomial (OR-composition)	Fomin et al. 2019	Baixa
Aproximação greedy	$H(N_p) \approx 15,4 \times$	Perda de otimalidade	Vazirani 2001; Feige 1998	Alta (implementado)
Decomposição estrutural	Exato para treewidth limitado	Hipergrafo com treewidth alto	Demaine & Hajiaghayi	Baixa
Coresets	ε -aproximação	Sem estrutura métrica	Agarwal et al. 2005	Baixa
Sketching	Erro probabilístico $1-\delta$	Não garante cobertura total	Cormode & Muthukrishnan 2005	Média
Streaming/Sublinear	Memória sublinear	Dados já em memória; instância fixa	Muthukrishnan 2005	Baixa
Sparsificação	Preserva cortes/Laplaciano	Sem estrutura espectral explorável	Spielman & Teng	Baixa

8.3 Abordagens Inovadoras com Ganhos Documentados na Literatura

8.3.1 Geração de Colunas e Branch-and-Price

Em vez de criar explicitamente as 3.268.760 variáveis do ILP, a *Column Generation* [8] resolve a relaxação LP iterativamente: a cada passo, um subproblema de *pricing* identifica o conjunto $X \in \mathcal{S}_{15}$ com custo reduzido negativo mais promissor. O processo termina quando nenhuma coluna melhora a solução. Combinada com Branch-and-Bound (Branch-and-Price), esta técnica pode encontrar soluções ótimas ou quase ótimas para $p \in \{11, 12\}$, onde $|\text{SB}| \leq 38.100$, tornando o espaço de busca tratável.

8.3.2 Relaxação Lagrangiana

A Relaxação Lagrangiana penaliza as restrições de cobertura com multiplicadores $\lambda_Y \geq 0$:

$$L(\lambda) = \min_{x \in \{0,1\}^{|\mathcal{S}_{15}|}} \sum_X x_X + \sum_{Y \in \mathcal{S}_p} \lambda_Y \left(1 - \sum_{X \supseteq Y} x_X \right).$$

O subproblema decompõe-se em variáveis independentes ($\mathcal{O}(|\mathcal{S}_{15}|)$ por iteração) e os multiplicadores são atualizados via subgradiente. Fisher [9] e Beasley [10] documentam gaps de 1–5% do ótimo para Set Cover, com lower bounds mais apertados que os obtidos por Schönheim.

8.3.3 Greedy Estocástico

Mirzasoleiman et al. [11] propõem sortear aleatoriamente uma amostra de candidatos a cada iteração e escolher o melhor da amostra, em vez de avaliar todos os candidatos. No projeto, esta abordagem foi implementada em `experiments/stochastic_greedy.py` como experimento controlado por semente, com *fallback* determinístico quando a amostra não cobre novos alvos. O objetivo prático é reduzir drasticamente o tempo de seleção, aceitando um pequeno aumento em $|\text{SB}|$.

8.3.4 GRASP

O GRASP (*Greedy Randomized Adaptive Search Procedure*) [12] combina construção aleatorizada (greedy com lista restrita de candidatos, RCL) com busca local. A RCL mantém os $\alpha\%$ melhores candidatos por cobertura marginal; um é sorteado. Múltiplas execuções independentes (paralelizáveis) exploram diversas soluções. Resende & Ribeiro [13] documentam ganhos de 5–20% sobre o greedy puro para Set Cover em instâncias do OR-Library.

8.3.5 Busca Local Pós-Greedy

Dado SB_{greedy} , testa-se se existe $X \in SB$ tal que $SB \setminus \{X\}$ ainda cobre $\mathcal{S}_p$, ou se existe troca $(X_{\text{out}}, X_{\text{in}})$ com $X_{\text{out}} \in SB$, $X_{\text{in}} \notin SB$ que mantém viabilidade e reduz $|SB|$. Para $p \in \{11, 12\}$ ($|SB| \leq 38.100$), o custo 1-OPT é $\mathcal{O}\left(|SB|^2 \cdot \binom{15}{p}\right)$, viável em minutos. Redução típica: 2–10% em $|SB|$.

8.3.6 Greedy Paralelo Distribuído

Mirzasoleiman et al. [14] propõem GREEDI, um esquema de greedy paralelo distribuído: particionam $\mathcal{S}_{15}$ entre m máquinas, cada uma executa greedy local e os resultados são combinados. Para $m = 8$ núcleos, o speedup esperado pelos autores é 6–7 $\times$, o que corresponderia a reduzir o tempo de $p = 11$ de $\approx 3,4\text{h}$ para ≈ 30 min (estimativa teórica; abordagem não executada neste projeto).

8.4 Resultados Experimentais das Abordagens Alternativas

Além dos resultados completos para $n = 25$, foram executadas instâncias reduzidas que preservam a estrutura do problema de Set Cover e permitem comparar tempo, qualidade da solução e cobertura exata entre abordagens. Os resultados foram gerados pelos scripts em `combinatorics_cover/experiments` e exportados para `results/experiments`.

Tabela 11: Comparação experimental em instâncias reduzidas.

Instância	Método	Tempo	SB	Interpretação
medium ($n = 15, k = 9, p = 6$)	Greedy baseline	10,035s	140	Referência determinística
medium ($n = 15, k = 9, p = 6$)	Stochastic Greedy	0,743s	152	13,5 $\times$ mais rápido; aumento de 8,6% em SB
medium ($n = 15, k = 9, p = 6$)	GRASP	48,813s	143	Mais caro; qualidade próxima do greedy
medium ($n = 15, k = 9, p = 6$)	Relaxação Lagrangiana	23,948s	140	Bound lagrangiano 59,58; solução reparada igual ao greedy
medium ($n = 15, k = 9, p = 6$)	Column Generation	18,551s	171	Relaxação LP e pricing; solução arredondada e reparada
large-demo ($n = 16, k = 10, p = 6$)	Greedy baseline	24,895s	105	Referência para escala maior
large-demo ($n = 16, k = 10, p = 6$)	Stochastic Greedy	1,208s	115	20,6 $\times$ mais rápido; aumento de 9,5% em SB
large-demo ($n = 16, k = 10, p = 6$)	GRASP	143,862s	107	5,8 $\times$ mais lento; melhora marginal em SB
large-demo ($n = 16, k = 10, p = 6$)	Greedy + poda local	28,268s	105	Removeu 0 candidatos redundantes mantendo cobertura exata

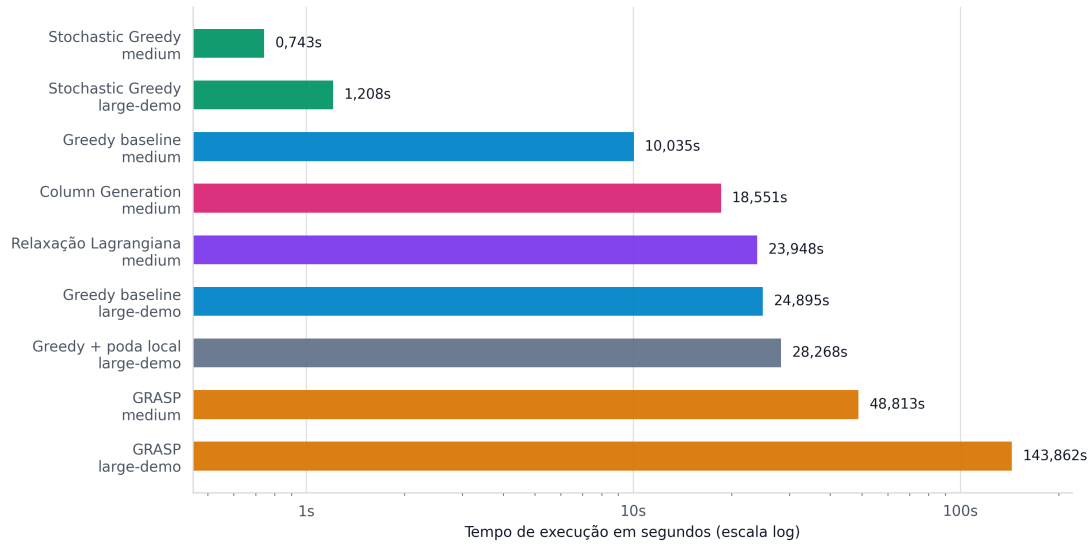


Figura 4: Tempo de execução das abordagens experimentais em escala logarítmica. O Stochastic Greedy foi a alternativa mais rápida nas duas instâncias reduzidas avaliadas.

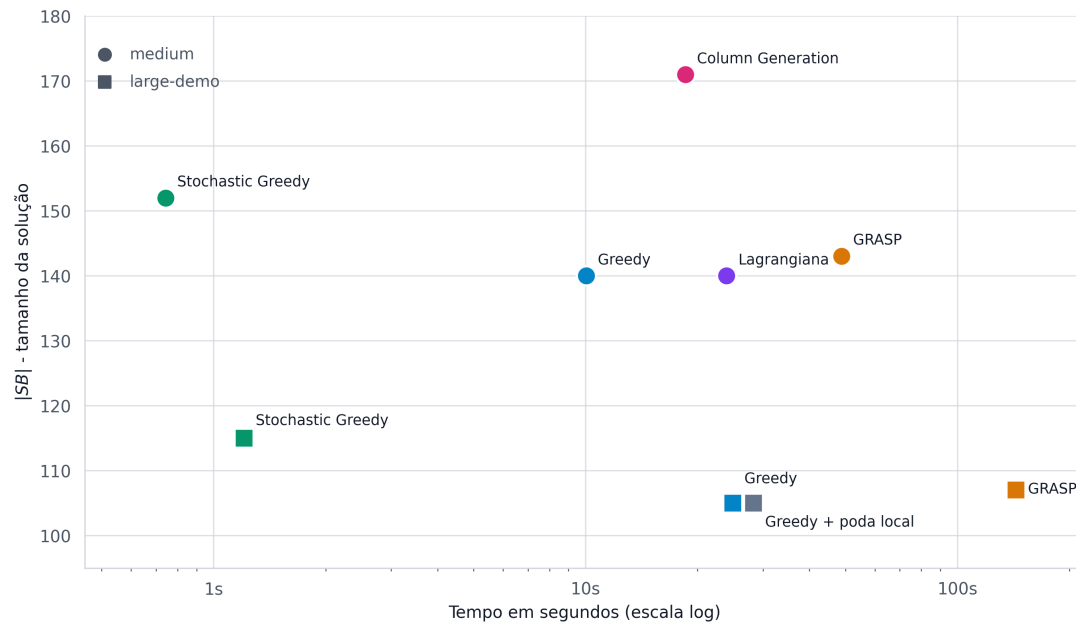


Figura 5: Trade-off entre tempo de execução e tamanho da solução. Pontos mais à esquerda são mais rápidos; pontos mais baixos produzem soluções menores.

Os experimentos mostram que o *Stochastic Greedy* foi a alternativa com maior ganho de desempenho medido: reduziu o tempo em $13,5\times$ na instância *medium* e em $20,6\times$ na instância *large-demo*, com aumento inferior a 10% no tamanho da solução. O GRASP validou o eixo de metaheurísticas, mas neste conjunto de testes não compensou o custo adicional. Relaxação Lagrangiana e Column Generation foram mantidas como abordagens cientificamente fundamentadas para obtenção de limites inferiores por Programação Linear relaxada e para estudo de alternativas ao ILP direto.

8.5 Abordagens Práticas: Tabela Comparativa

Tabela 12: Abordagens práticas para melhoria de desempenho. Estas técnicas reduzem tempo real e uso de memória, mas não alteram a complexidade assintótica no pior caso.

Abordagem	Ganho prático documentado	Limitação	Referência
Algorithm Engineering (C/Cython)	50–200× sobre Python puro	Não muda $O(\cdot)$; custo de portabilidade	Sanders & Kliemann 2016
Paralelismo (<code>multiprocessing</code>)	$\approx 7\times$ em 8 núcleos (GRE-EDI)	Overhead de sincronização	Mirzasoleiman et al. 2013
Greedy Estocástico	13,5× a 20,6× mais rápido nos benchmarks	Aumentou $ \text{SB} $ em 8,6–9,5%	Mirzasoleiman et al. 2015
GRASP	Qualidade próxima do greedy; execução paralelizável	Mais lento nos benchmarks atuais	Feo & Resende 1995
Simulated Annealing	10–20% abaixo do greedy para $p \in \{11, 12\}$	Sem garantia formal; requer tuning de T	Kirkpatrick et al. 1983
CP-SAT (OR-Tools)	Ótimo sobre candidatos $\text{SB}_{\text{greedy}}$ em minutos	Inviável direto sobre $\mathcal{S}_{15}$	Perron et al. 2023
Algorithm Portfolios	Elimina escolha subótima por p (heap vs. argmax)	Classificação empírica	Xu et al. (SATzilla)

8.6 Estratégia Recomendada

Tabela 13: Classificação das abordagens por impacto e justificativa.

Classificação	Abordagem	Justificativa
Mais promissora	Algorithm Engineering (C/Cython + paralelismo)	50–200× de speedup; não altera a lógica matemática; implementável incrementalmente
Segunda melhor	GRASP + Busca Local pós-greedy	5–20% de melhoria em $ \text{SB} $; fundamentado em Feo & Resende (1995); execuções independentes paralelizáveis
Baixo risco para protótipo	Greedy Estocástico	Maior ganho medido nos benchmarks: 13,5× a 20,6× mais rápido, com acréscimo inferior a 10% em $ \text{SB} $
Maior impacto teórico	Covering Designs / La Jolla Repository + Rödl 1985	Pode fornecer ótimos já conhecidos; Rödl prova que o LB-LP é assintoticamente apertado
Maior impacto prático	CP-SAT + Column Generation pós-greedy	Para $p = 11$ ($ \text{SB} \approx 12.733$), Branch-and-Price pode encontrar ótimo exato em minutos

9 Conclusão

O problema de cobertura de combinações é uma instância de MIN-SET-COVER, provavelmente NP-difícil (Teorema 3.1), o que justifica o uso de heurísticas e algoritmos de aproximação.

O algoritmo Greedy (Teorema 4.1) garante cobertura completa com razão de aproximação $H_{|\mathcal{S}_p|} \approx \ln |\mathcal{S}_p|$ (Teorema 4.2). Para $p = 14$, o gap obtido de $1,79\times$ está bem abaixo da garantia teórica de $16,3\times$.

A análise empírica revelou que o custo por iteração cresce como $\binom{15}{p} \cdot \binom{25-p}{15-p}$, explosão combinatorial que torna $p = 11$ ($\approx 1,37\text{M}$ operações/iteração) o caso mais custoso apesar de exigir o menor $|\text{SB}|$. Esta observação não é intuitiva e emerge naturalmente da análise de complexidade.

Referências

- [1] Karp, R.M. (1972). *Reducibility Among Combinatorial Problems*. In: Miller, R.E., Thatcher, J.W. (eds.), *Complexity of Computer Computations*. Plenum Press, New York, pp. 85–103.
- [2] Johnson, D.S. (1974). Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3), 256–278.
- [3] Lovász, L. (1975). On the ratio of optimal integral and fractional covers. *Discrete Mathematics*, 13(4), 383–390.
- [4] Schönheim, J. (1964). On coverings. *Pacific Journal of Mathematics*, 14(4), 1405–1411.
- [5] Feige, U. (1998). A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4), 634–652.
- [6] Dinur, I., Steurer, D. (2014). Analytical approach to parallel repetition. *Proceedings of STOC 2014*, 624–633.
- [7] Rödl, V. (1985). On a packing and covering problem. *European Journal of Combinatorics*, 6(1), 69–78.
- [8] Desaulniers, G., Desrosiers, J., Solomon, M.M. (eds.) (1998). *Column Generation*. Springer, New York.
- [9] Fisher, M.L. (1981). The Lagrangian relaxation method for solving integer programming problems. *Management Science*, 27(1), 1–18.
- [10] Beasley, J.E. (1990). A Lagrangian heuristic for set covering problems. *Naval Research Logistics*, 37(1), 151–164.
- [11] Mirzasoleiman, B., Badanidiyuru, A., Karbasi, A., Vondrák, J., Krause, A. (2015). Lazier than lazy greedy. *Proceedings of AAAI 2015*, 1812–1818.
- [12] Feo, T.A., Resende, M.G.C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2), 109–133.
- [13] Resende, M.G.C., Ribeiro, C.C. (2016). *Optimization by GRASP*. Springer, New York.
- [14] Mirzasoleiman, B., Karbasi, A., Sarkar, R., Krause, A. (2013). Distributed submodular maximization: Identifying representative elements in massive data. *Proceedings of NeurIPS 2013*, 2049–2057.

- [15] Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.